

Movies ELK Plateforme

Document de synthèse

Avril 2026

Réalisé par : SADI Celine

Table des matières

1. Contexte et périmètre
2. Architecture et environnement
3. Documentation des données et nettoyage
 - 3.1 Anomalies observées
 - 3.2 Règles de nettoyage appliquées dans Logstash
 - 3.3 Mesure d'impact avant/après
4. Modelisation Elasticsearch
 - 4.1 Mapping de movies_clean
 - 4.2 Analyzer personnalisé movies_analyzer
5. Requêtes et analyses
6. Dashboard Kibana et lecture métier
7. Gestion de projet et collaboration
8. Bilan, limites et améliorations
9. Conclusion

1.Contexte et périmètre

Ce projet consiste à construire une plateforme d'analyse de données cinématographiques, donc un mini moteur de recherche de film avec la stack ELK. La base de données utilisée est le TMDb Movies Dataset (Kaggle) contenant environ 770 000 films.

L'objectif est de mettre en place un pipeline complet allant de l'ingestion des données brutes jusqu'à la visualisation des données et la recherche depuis le mini moteur de recherche développé.

Toutes les fonctionnalités obligatoires ont été implémentées : stack Docker Compose, deux index Elasticsearch, pipeline Logstash, mapping explicite, 12 requêtes dont 5 bool, un dashboard Kibana avec 7 visualisations et moteur de recherche avec Streamlit (une bibliothèque python open source pour la création et le partage d'application). Le projet a été réalisé individuellement avec respect du Gitflow.

2.Architecture et environnement

La stack est entièrement déployée via Docker Compose avec 5 services : Elasticsearch, Kibana, Logstash, PostgreSQL et Jupyter.

Service	Rôle
Elasticsearch	Moteur de stockage et d'indexation des données. Il reçoit les documents de Logstash, applique le mapping et répond aux requêtes de recherche et d'agrégation.
Kibana	Interface graphique connectée à Elasticsearch. Permet de créer des visualisations, des dashboards analytiques et d'exécuter des requêtes via le Dev Tools.
Logstash	Pipeline d'ingestion et de transformation. Il lit le fichier CSV, parse les colonnes, nettoie les données et les envoie vers les index Elasticsearch.
PostgreSQL	Base de données relationnelle disponible pour les notebooks Jupyter. Utilisée comme source alternative de données structurées.
Jupyter	Environnement de développement interactif. Permet d'exécuter les requêtes Elasticsearch en Python et de produire les analyses documentées.

Le flux de données suit le chemin suivant :

movies.csv -> Logstash -> movies_raw + movies_clean -> Kibana + Streamlit

Chaque service dispose d'un healthcheck pour garantir la disponibilité avant démarrage des services dépendants. Le fichier CSV est monté en lecture seule dans le conteneur Logstash.

3.Documentation des données et nettoyage

3.1 Anomalies observées :

- Dates aberrantes : certains films ont des dates de sortie en 2090 ou 2099, dues à des erreurs de saisie dans TMDB
- Valeurs nulles : les champs budget, revenue et runtime contiennent des zéros pour les films non documentés
- Types incorrects : tous les champs CSV sont lus comme du texte, les conversions sont obligatoires
- Doublons : plusieurs relances de Logstash ont généré 1 515 892 documents au lieu de 770 000

3.2 Règles de nettoyage appliquées dans Logstash :

Règle	Implementation
Suppression header CSV	if [id] == "id" { drop {} }
Suppression films sans titre	if ![title] or [title] == "" { drop {} }
Conversion types numériques	mutate { convert => { popularity => float, vote_average => float... } }
Parsing de la date	date { match => ["release_date", "yyyy-MM-dd"] }
Prévention des doublons	document_id => "%{id}"

3.3 Mesure d'impact avant/après :

La différence de 108 000 documents correspond aux films supprimés lors du nettoyage (sans titre, doublons, invalides).

Indicateur	movies_raw	movies_clean
Nombre de documents	770 000	662 083
Types corrects	Non	Oui
Doublons	Oui	Non
Champs techniques	Présents	Supprimés

4. Modelisation Elasticsearch

4.1 Mapping de movies_clean :

Un mapping explicite contrôle le type de chaque champ. Les champs text (title, overview) sont analysés pour la recherche full-text. Les champs keyword (genres, original_language, status) permettent les filtres exacts et les agrégations Kibana. Les champs numériques (float, integer, long) permettent les calculs et tris.

4.2 Analyzer personnalisé movies_analyzer :

L'analyzer applique 4 transformations en chaîne : tokenizer standard (découpé en mots), lowercase (insensible à la casse), stop (supprime the/a/an...), snowball (running -> run). Il est appliqué sur title et overview pour une recherche pertinente et tolérante aux variations.

5. Requêtes et analyses

12 requêtes commentées ont été produites dans le notebook Jupyter, dont 5 requêtes bool couvrant des cas métier concrets :

N	Requête bool	Cas d'usage
5	Films d'action anglais mieux notés	must + filter : recommandation de films d'action américains
6	Films populaires récents	must + double filter : page d'accueil streaming
7	Grands flops (budget > 100M\$, note < 5)	must + filter : analyse échecs des films
8	Cinéma mondial hors anglais/français	must + must_not : section cinéma du monde
9	Recherche multi-champs sur un thème	should + minimum_should_match : moteur thématique

Les 7 autres requêtes couvrent : comptage total, inspection document, recherche full-text, top notes, agrégation par langue, note moyenne par langue, évolution par année.

6. Dashboard Kibana et lecture métier

Le dashboard Movies Analytics contient 7 visualisations produites avec Kibana Lens :

- Répartition par langue (camembert) : l'anglais domine à 51%, reflétant la domination d'Hollywood
- Evolution par année (courbe) : la production explose après 2000 avec le numérique
- Top 10 films populaires (barres) : les franchises Marvel dominent le classement TMDB
- Distribution des notes (histogramme) : concentration entre 5 et 7 sur 10
- Budget moyen par langue (barres) : les productions anglophones ont le budget le plus élevé
- Note moyenne par langue (barres) : certaines cinématographies peu connues obtiennent de hautes notes
- Top 10 genres (barres) : le documentaire et le drame sont les plus représentés

7. Gestion de projet et collaboration

Le projet a été réalisé individuellement avec un Gitflow strict : branches main, dev et 4 branches feature (bootstrap-stack, queries-notebook, search-engine, documentation). Chaque feature a fait l'objet d'une Pull Request vers dev avant merge. Aucun push direct sur main ou dev.

Un fichier docs/planning_poker.md documente le backlog en échelle Fibonacci avec les estimations par feature. Les features les plus complexes (nettoyage + mapping, dashboard Kibana) ont été estimées à 8 points.

8. Bilan, limites et améliorations

La plateforme est fonctionnelle et reproductible sur une machine vierge. Les principales difficultés résolues : doublons Logstash (document_id), dates aberrantes (filtre range), incompatibilité widgets Jupyter (remplacement par Streamlit).

Axes d'amélioration : normaliser le champ genres, nettoyer les budgets à zéro dans Logstash, ajouter la pagination dans le moteur de recherche, comparer plusieurs analyzers pour mesurer leur impact sur la pertinence des résultats.

9. Conclusion

Ce projet a été réalisé intégralement de manière individuelle, ce qui a représenté un défi considérable au regard du volume de travail attendu pour une équipe de quatre personnes. Malgré cette contrainte, l'ensemble des livrables obligatoires ont été produits : une stack ELK opérationnelle, deux index Elasticsearch structures, un pipeline Logstash documente, un dashboard Kibana analytique, 12 requêtes commentées et un moteur de recherche fonctionnel.

Ce projet a été une véritable opportunité d'apprentissage. J'ai acquis une maîtrise concrète de la stack ELK dans un contexte de données réelles à grande échelle, comprenant les enjeux de la qualité des données et de leur nettoyage, et mis en pratique des méthodes de gestion de projet professionnelles comme le Gitflow et le planning poker. La confrontation aux problèmes réels, loin des exemples simplifiés du cours, m'a permis de développer une vraie autonomie technique et une meilleure compréhension des outils utilisés.